

Differential-based Attacks on Round-reduced BAKSHEESH

Yong Liu^{1,2}, Siwei Sun^{1,2,*}, Zejun Xiang³, Rui Xu⁴, Xiutao Feng⁵, and
Xiangyong Zeng⁴

¹ School of Cryptology, University of Chinese Academy of Sciences, Beijing, China

² State Key Laboratory of Cryptology, P.O. Box 5159, Beijing, China

{liuyong,sunsiwei}@ucas.ac.cn

³ School of Cyber Science and Technology, Hubei University, Wuhan, China

xiangzejun@hubu.edu.cn

⁴ Key Laboratory of Intelligent Sensing System and Security, Ministry of Education;

Hubei Key Laboratory of Applied Mathematics, Faculty of Mathematics and

Statistics, Hubei University, Wuhan, China

13163354821@163.com, xzeng@hubu.edu.cn

⁵ State Key Laboratory of Mathematical Sciences, Academy of Mathematics and

Systems Science, Chinese Academy of Sciences, Beijing, China

fengxt@amss.ac.cn

Abstract. BAKSHEESH is an efficient lightweight block cipher that features both GIFT-like and ZORRO-like structures. In this paper, we first construct Mixed-Integer Linear Programming models that constrain the Hamming weights of the input and output differences/masks of the middle part, enabling the efficient search for advantageous differential-linear (DL) and boomerang distinguishers under limited computational resources. Furthermore, by exploiting decision tree techniques and the two-round differential propagation property of BAKSHEESH, we parameterize the number of guessed key bits and the logarithm of the number of possible plaintext/ciphertext differences, and subsequently build MILP models to identify the best DL and boomerang key-recovery attacks under the stated constraints. As a result, we identify the first 17-round DL distinguisher and 16-round boomerang distinguisher for BAKSHEESH, yielding the first 19-round DL attack and 18-round boomerang attack with time complexities of $2^{126.97}$ and $2^{126.48}$, respectively. In addition, we construct the first 20-round chosen-ciphertext differential attack with a time complexity of $2^{122.59}$, improving the best previously known differential attack by one round. Overall, our results provide new insights into the security of BAKSHEESH against differential-based cryptanalysis.

Keywords: BAKSHEESH · MILP · Differential-based attack · Key-recovery attack

1 Introduction

The security of symmetric-key primitives is fundamentally evaluated based on their resistance to all known cryptanalytic techniques, among which differential cryptanalysis [1] and linear cryptanalysis [2] are two of the most classical and

effective methods. Given the effectiveness of differential and linear cryptanalysis, resistance against long-round differential characteristics and linear trails has become one of the fundamental design principles of modern block ciphers.

In fact, short-round differential characteristics and linear trails can still be combined to construct advanced cryptanalytic techniques, such as boomerang cryptanalysis [3] and differential-linear (DL) cryptanalysis [4]. Originally, boomerang cryptanalysis [3] and DL cryptanalysis [4] both partition the target cipher into two parts. However, extensive studies have shown that the assumption of independence between these two parts does not always hold in practice. To address this issue, Dunkelman *et al.* [5] introduced the boomerang framework, which decomposes the cipher into three parts, namely an upper differential part, a middle part, and a lower differential part. Assuming that the probabilities associated with these three parts are p_B , r_B , and q_B , respectively, the total probability of the resulting boomerang distinguisher can be evaluated as $p_B^2 r_B q_B^2$. Similarly, Bar-On *et al.* [6] presented a DL distinguisher divided into a differential part, a middle part, and a linear part. Assuming that the differential part has a probability of p_{DL} , and the middle and linear parts have correlations of r_{DL} and q_{DL} , respectively, the overall correlation of the distinguisher can be evaluated as $p_{DL} r_{DL} q_{DL}^2$.

In general, constructing valid distinguishers is a crucial step in cryptanalysis. Currently, the most efficient approach relies on automated tools such as Mixed-Integer Linear Programming (MILP), Boolean Satisfiability (SAT), and Constraint Programming (CP), such as [7,8,9]. A major bottleneck in the automated search for boomerang and DL distinguishers lies in accurately evaluating the transition probability or correlation of the middle part, particularly when the middle part spans multiple rounds, where directly characterizing such propagation behaviors remains highly non-trivial. Consequently, Delaune *et al.* [10] employed a suite of transition tables (DDT, BCT, EBCT, UBCT, and LBCT) to model multi-round boomerang probabilities, while Bellini *et al.* [11] introduced continuous difference propagation for correlation estimation. Recognizing the limitations in applicability and scalability of existing models, Hadipour *et al.* [12,13] proposed a general estimation framework based on the number of shared active S-boxes or active bits in forward and backward trails.

However, such automated models, which rely on indirect characterizations of correlation or probability, cannot guarantee the validity of the resulting distinguishers, thereby requiring empirical verification. This limitation has motivated studies from diverse theoretical perspectives, including the algebraic perspective [14], the truncated differential perspective [15], and Beyne’s geometric perspective [16]. Nevertheless, these frameworks remain difficult to integrate into automated models. Consequently, several works have focused on heuristic optimizations to improve search efficiency. For instance, Chen *et al.* [17] accelerated the search by pruning low-correlation output mask bits in the middle rounds. More recently, Gong *et al.* [18] exploited the hourglass(-like) structure, which is commonly observed in optimal DL distinguishers for ARX ciphers, to significantly reduce the search space.

Since the ultimate objective of cryptanalysis is key recovery, it is more desirable to integrate the key-recovery phase into the model and directly search for optimal key-recovery attacks rather than distinguishers alone. Following this idea, Hadipour *et al.* [19] unified the distinguisher search model with the generic complexity formula for impossible differential key-recovery attacks [20], enabling the direct search for attacks with optimal time complexity. This observation has motivated the development of several key-recovery frameworks for differential-based cryptanalysis [21,22,23], enabling the key-recovery phase to be naturally incorporated into automated models for searching attack configurations through generic complexity formulas.

However, although generic key-recovery frameworks can be readily incorporated into automated models, they often fail to fully capture how the cryptographic properties of non-linear components dictate the key-recovery process. Consequently, the attack configurations identified by such models are not necessarily optimal. In fact, several studies have exploited the algebraic or propagation properties of non-linear layers to further reduce attack complexity, primarily by minimizing the number of guessed subkey bits and enabling the early filtering of wrong pairs. For example, Broll *et al.* [24] proposed a generic key-recovery framework that exploits the cryptographic properties of S-boxes by representing these properties as a special type of decision tree, which is crucial for finding fine-grained guessing strategies. This framework was subsequently applied to the DL cryptanalysis of Serpent [25,26]. Similarly, Wang *et al.* [27] and Qiao *et al.* [28] proposed a dynamic key-guessing technique for Simon and Simeck by exploiting the bit conditions of the extended rounds, thereby reducing both the number of subkey guesses and the required data complexity. Liu *et al.* [29] improved the boomerang attacks on 11-round Deoxys-BC-256 and 13-round Deoxys-BC-384 by optimizing key-guessing strategies and precomputation techniques. Consequently, developing automated models that leverage the fine-grained cryptographic properties of non-linear components to search for the best key-recovery attack configurations under the stated constraints is of great significance.

Our Contributions. In this paper, we develop an automated framework that first searches for DL and boomerang distinguishers, and then automatically identifies the best attack configurations under the stated constraints, based on these obtained distinguishers. The main contributions are as follows:

Two-round differential propagation property of BAKSHEESH. Since the permutation of BAKSHEESH is identical to that of GIFT-128 [30], its permutation layer exhibits the quotient-remainder ($\mathcal{QR}$) group structure. Specifically, for a given input difference over a specific set of four consecutive S-boxes, two rounds of propagation (excluding the permutation layer in the second round) map the resulting difference exactly to the outputs of four non-consecutive S-boxes. Consequently, knowing the exact input difference enables an exhaustive enumeration of all feasible 16-bit output differences after two rounds. Based on this property, we derive all possible ciphertext differences from the output difference of the distinguisher and discard invalid data pairs in advance, thereby reducing the time complexity of key-recovery attacks.

MILP-based models for searching distinguishers and attacks. Motivated by the observation that the best-known DL and boomerang distinguishers typically exhibit an hourglass-like structure, we build MILP models that constrain the Hamming weights of the input and output differences/masks of the middle rounds to at most two or three, thereby enabling the efficient identification of advantageous distinguishers for large-state primitives within practical computational limits. Furthermore, by exploiting the decision tree technique together with the two-round differential propagation property of BAKSHEESH, we parameterize the number of guessed key bits in the extended rounds and the logarithm of the number of possible plaintext/ciphertext differences, and construct MILP models to identify the best DL and boomerang key-recovery attack configurations under the stated constraints.

Best-known differential, DL, and boomerang attacks on BAKSHEESH.

We obtain the first 17-round DL distinguisher and the first 16-round boomerang distinguisher for BAKSHEESH, from which we further derive the first 19-round DL attack and the first 18-round boomerang attack, with time complexities of $2^{126.97}$ and $2^{126.48}$, respectively. In addition, we construct the first 20-round chosen-ciphertext differential attack on BAKSHEESH with a time complexity of $2^{122.59}$, improving the best previously known differential attack by one round. A detailed comparison with previous results is provided in Table 1.

Organization. Section 2 reviews DL and boomerang cryptanalysis, as well as the decision tree technique. Section 3 introduces BAKSHEESH and its relevant properties. Section 4 presents the automated models for distinguishers and key-recovery attacks. Section 5 reports the resulting attacks on BAKSHEESH. Finally, Section 6 concludes the paper. The full version of this paper and the source code of our tool are publicly available at <https://github.com/CRYPTO12138-coder/Differential-based-Attacks-on-Round-reduced-BAKSHEESH>.

Table 1: Summary of attack results on BAKSHEESH

Attack Type	Rounds	Time	Data	Memory	Reference
Integral	15	2^{127}	2^{127}	2^{23}	[31]
Differential	19	2^{121}	2^{122}	2^{64}	[31]
Linear	21	$2^{127.67}$	2^{120}	2^{12}	[31]
Linear	24	$2^{125.25}$	2^{125}	2^{85}	[32]
Differential-Linear	19	$2^{126.97}$	$2^{120.94}$	$2^{120.94}$	Subsection 5.1
Boomerang	18	$2^{126.48}$	$2^{110.09}$	$2^{110.09}$	Subsection 5.2
Differential	20	$2^{122.59}$	2^{122}	2^{104}	Appendix A

2 Background

2.1 Differential-based Cryptanalysis

Differential-Linear Attack. Consider an r -round DL distinguisher $\Delta_{in} \rightarrow \Gamma_{out}$ for the target cipher E , where the differential part has probability p_{DL} , and the middle and linear parts have correlations r_{DL} and q_{DL} , respectively. The overall

correlation of the distinguisher is given by $p_{\text{DL}} r_{\text{DL}} q_{\text{DL}}^2$. A full DL attack is then constructed by extending the distinguisher by r_b rounds backward and r_f rounds forward. Let V_b be the space spanned by all possible plaintext differences, where $m_b = \log_2 |V_b|$. Let k_b (k_f) be the number of guessed subkey bits involved in the backward (forward) extension. The corresponding key-recovery procedure is detailed in [25,26] and is summarized in Algorithm 3 and illustrated in Figure 4, both in Appendix C of the full version of this paper.

Complexity Evaluation. According to the statistical model of [33], the number of plaintext-ciphertext pairs required for the attack can be estimated as

$$N \approx \frac{\Phi^{-1}(1 - 2^{-a-1})^2 - \Phi^{-1}(1 - P_s/2)^2}{(\text{ELP} - 2^{-n})\Phi^{-1}(1 - P_s/2)^2}, \quad (1)$$

where a denotes the advantage, P_s is the success probability, Φ is the cumulative distribution function of the standard normal distribution, and $\text{ELP} = (p_{\text{DL}} r_{\text{DL}} q_{\text{DL}}^2)^2$. Thus, the data complexity of the attack is $D = 2N$. Since the memory overhead is primarily dominated by the storage of the required plaintext pairs, the memory complexity is evaluated as $M = D$ blocks.

The overall time complexity is dominated by the key guessing procedure. The number of iterations of the outer loop in the key recovery algorithm is 2^{k_b} . For each iteration, the linear-side key recovery is performed. Using the Fast Fourier Transform (FFT) technique of [34], this step costs $\mathcal{O}(N) + \mathcal{O}(2(\rho_M + \rho_A k_f) \cdot 2^{2k_f})$, where ρ_A and ρ_M denote the costs of n -bit (n being the state size of the target cipher) addition and multiplication, respectively. Following the frameworks established in [25,26] and employing the evaluation model from [33], the total time complexity of the DL key-recovery attack is estimated to be

$$T \approx \underbrace{N}_{\text{Data Collection}} + \underbrace{2^{k_b} \left(N\varepsilon + 2(\rho_M + \rho_A k_f) \cdot 2^{2k_f} \right)}_{\text{Guess-and-Filter}} + \underbrace{2^{k-a}}_{\text{Exhaustive Search}}, \quad (2)$$

where ε is the ratio of the cost of partial encryption to that of full encryption.

Boomerang Attack. Consider an r -round boomerang distinguisher $\Delta_{in} \rightarrow \Gamma_{out}$, where the upper differential, middle, and lower differential parts have probability p_B , r_B , and q_B , respectively. Its overall probability is $p_B^2 r_B q_B^2$. A full boomerang attack is constructed by extending this distinguisher by r_b rounds backward and r_f rounds forward, respectively. Let V_b (V_f) be the space spanned by all possible plaintext (ciphertext) differences, where $m_b = \log_2 |V_b|$ ($m_f = \log_2 |V_f|$). Let k_b (k_f) be the number of guessed subkey bits involved in the backward (forward) extension. The corresponding key-recovery procedure is detailed in [38,39] and is summarized in Algorithm 4 and illustrated in Figure 5, both in Appendix C of the full version of this paper.

Complexity Evaluation. For a boomerang distinguisher with a total probability of $\text{Pr}^2 = p_B^2 r_B q_B^2$, the required number of right quartets is approximately

$\mathcal{O}(\text{Pr}^{-2})$. Accordingly, the data complexity needed to generate a sufficient number of quartets is bounded by

$$D = \sqrt{s} \cdot 2^{n/2+1-m_b} / \text{Pr} \cdot 2^{m_b} = \sqrt{s} \cdot 2^{n/2+1} / \text{Pr}, \quad (3)$$

where s denotes the expected number of surviving quartets. The memory complexity is dominated by D blocks to store the plaintext pairs and 2^{k_f} counters for the key recovery phase. Furthermore, following the statistical model of Selçuk [35], the success probability of a boomerang attack achieving an a -bit advantage is estimated as $P_s = \Phi\left(\frac{\sqrt{s S_N} - \Phi^{-1}(1-2^{-a})}{\sqrt{S_N+1}}\right)$, where $S_N = \text{Pr}^2 / 2^{-n}$ denotes the signal-to-noise ratio, $\Phi(\cdot)$ is the cumulative distribution function of the standard normal distribution.

Following the frameworks established in [38,39], the overall time complexity of the boomerang key-recovery attack can be expressed as

$$T \approx \underbrace{\sqrt{s} 2^{k_b+n/2+1} / \text{Pr}}_{\text{Data Collection}} + \underbrace{s 2^{k_b-n+2m_f} / \text{Pr}^2 \cdot \epsilon}_{\text{Guess-and-Filter}} + \underbrace{2^{k-a}}_{\text{Exhaustive Search}}, \quad (4)$$

where s denotes the expected number of surviving quartets, ϵ represents the cost of processing each candidate during the Guess-and-Filter phase.

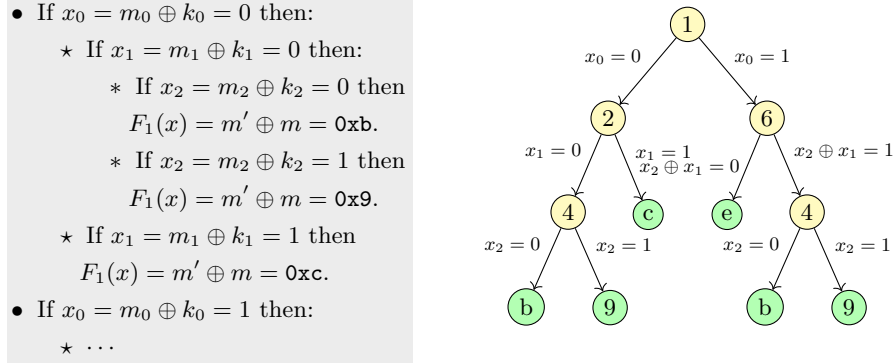
Since differential and boomerang attacks share the same statistical models and follow similar attack procedures, we omit their generic complexity formulas for brevity.

2.2 Decision Tree Technique

At ASIACRYPT 2021, Broll *et al.* [24] introduced a generic framework to systematically optimize the key-recovery guessing process by modeling an S-box (or selected components thereof) as a binary decision tree, proving that the minimum number of required subkey guesses is equivalent to the number of leaves in the resulting optimal tree.

Let the input and output of an S-box S be $x = m \oplus k$ and $y = S(x)$, respectively. Assuming that the output difference Δ of the S-box is fixed and known, the objective is to determine the counterpart m' for a given plaintext m such that the pair $(x, x') = (m \oplus k, m' \oplus k)$ satisfies the target output difference Δ while minimizing the number of guessed key bits of k . To formalize this transition, we define the function $F_\Delta(x) = S^{-1}(S(x) \oplus \Delta) \oplus x = x' \oplus x = m' \oplus m$. Consequently, once the value of $F_\Delta(x)$ is determined, the required plaintext m' can be directly deduced via $m' = F_\Delta(x) \oplus m$.

To clarify, we provide an example using the BAKSHEESH S-box S [36,37] (detailed in Subsection 3.1) under the output difference $\Delta = 0\mathbf{x}1$. Figure 1 depicts the decision tree formulated for F_1 , illustrating that the evaluation of $F_1(x)$ is contingent upon the successive guessing of subkey bits k_0 , k_1 , k_2 , and $k_2 \oplus k_1$. As a result, recovering m' requires only six key guesses, rather than sixteen in the conventional approach.

Fig. 1: Decision tree and its execution logic for $F_1(x)$

3 BAKSHEESH

3.1 Round Function of BAKSHEESH

BAKSHEESH is a lightweight block cipher proposed by Baksi *et al.* [36,37], which operates on a 128-bit block using a 128-bit secret key. Representing its internal state as $X = (x_{127}, x_{126}, \dots, x_0)$, the cipher executes a 35-round encryption procedure based on a Substitution-Permutation Network (SPN) structure. Specifically, each round comprises the following four operations.

SubCells. The 128-bit state is divided into 32 nibbles. Each nibble is transformed using the 4-bit S-box S defined as

$[0x3, 0x0, 0x6, 0xd, 0xb, 0x5, 0x8, 0xe, 0xc, 0xf, 0x9, 0x2, 0x4, 0xa, 0x7, 0x1]$.

PermBits. The PermBits layer relocates each bit at position i to position $P_L(i)$, namely $x_{P_L(i)} \leftarrow x_i, i \in \{0, 1, \dots, 127\}$. The permutation P_L is the same as that used in GIFT-128 [30].

AddRoundConstants. For each round r ($0 \leq r \leq 34$), a 6-bit round-dependent constant is injected into the state by XORing it with the bits located at positions 8, 13, 19, 35, 67, and 106.

AddRoundKey. In each round, a 128-bit round key $RK_r = (k_r^{127}, k_r^{126}, \dots, k_r^0)$ is XORed with the internal state. The master key $k = (k^{127}, k^{126}, \dots, k^0)$, which serves as RK_0 , is directly utilized as the whitening key. All subsequent round keys are derived by cyclically rotating the master key to the right. Specifically, the round key applied in round r is formally defined as $RK_r = k \ggg r, 0 \leq r \leq 35$.

Remark 1. An early version of BAKSHEESH was introduced in [36] and later formalized in [37]. The two versions differ only in the permutation P_L and the positions at which the round constants are applied. For consistency with existing third-party analyses, we adopt the description given in [36]. Furthermore, in Appendix D of the full version of this paper, we extend our proposed attacks to the version specified in [37].

3.2 Properties of the BAKSHEESH

For the BAKSHEESH S-box, the following properties can be exploited to facilitate the key-guessing process.

Proposition 1. *Assume that the output difference Δ of the BAKSHEESH S-box S is known. Then one can determine plaintext pairs (m, m') such that $S(m \oplus k) \oplus S(m' \oplus k) = \Delta$ involving at most 6 key guesses. Specifically, the number of key guesses N_{gk} is given by*

$$N_{gk} = \begin{cases} 0, & \text{if } \Delta = 0\mathbf{x}0 \text{ or } \Delta = 0\mathbf{x}\mathbf{f}, \\ 6, & \text{otherwise.} \end{cases}$$

Moreover, regardless of the value of Δ , the most significant bit of the key, k_3 , does not need to be guessed.

Proposition 2. *The algebraic normal form (ANF) of the inverse S-box S^{-1} of BAKSHEESH is given by*

$$\begin{cases} x_0 = (y_0 \oplus y_3)(y_1 \oplus y_2) \oplus y_2 \oplus y_3 \oplus 1, & x_1 = y_0y_2 \oplus y_3(y_0 \oplus y_2) \oplus y_0 \oplus y_1 \oplus y_3, \\ x_2 = y_0(y_1 \oplus y_2) \oplus y_0 \oplus y_1y_2 \oplus y_2 \oplus y_3, & x_3 = y_0 \oplus y_1 \oplus y_2. \end{cases}$$

Consequently, recovering the output bits (x_0, x_1, x_2, x_3) requires guessing the subkey-bit combinations $(k_2 \oplus k_1, k_3 \oplus k_2, k_2 \oplus k_0)$, $(k_2 \oplus k_0, k_3 \oplus k_1, k_2 \oplus k_1)$, $(k_1 \oplus k_0, k_3 \oplus k_2, k_3 \oplus k_1)$, and $(k_2 \oplus k_1 \oplus k_0)$, respectively.

Quotient-Remainder Group Structure. Since the permutation layer of BAKSHEESH is identical to that of GIFT-128, it also exhibits the quotient-remainder ($\mathcal{QR}$) group structure [30]. In particular, the outputs of four consecutive S-boxes in one round are mapped to the inputs of four non-consecutive S-boxes in the next round, allowing the S-boxes in two consecutive rounds to be viewed as a super S-box. Specifically, let $S_j^{A_j}$ ($0 \leq j \leq 7$) denote the input bits of the S-box layer, where $A_j = 0, 1, \dots, 15 + 16j$. Under the permutation P_L , $S_j^{A_j}$ is mapped precisely to $S_j^{B_j}$, where $B_j = \{0, 1, 2, 3, 32, 33, 34, 35, 64, 65, 66, 67, 96, 97, 98, 99\} + 4j$. Consequently, the 16 output bits in $S_j^{B_j}$ are distributed among the inputs of four distinct S-boxes.

4 MILP Models for Searching Differential-Linear and Boomerang Attacks

4.1 MILP Models for Searching Distinguishers

We adopt the hourglass(-like) structural search technique proposed by Gong *et al.* [18] and generalize it to the automated construction of boomerang distinguishers. Taking the search for DL distinguishers as an example, our approach

can be divided into three phases and follows a divide-and-conquer paradigm. First, we independently enumerate all r_d -round differential characteristics with an output Hamming weight of at most three (Δ , probability p_{DL}) and all r_ℓ -round linear approximations with an input mask Hamming weight of at most two (Γ , correlation q_{DL}). Next, we experimentally evaluate the r_m -round middle-part correlations r_{DL} for all $\left(\sum_{i=1}^3 \binom{128}{i}\right) \times \left(\sum_{i=1}^2 \binom{128}{i}\right)$ combinations of Δ and Γ to construct the DL distinguishers. Finally, the DL distinguisher candidate best found under the above constraints is determined by maximizing the absolute correlation metric $|p_{DL}r_{DL}q_{DL}^2|$.

Determination of Parameters r_d , r_m , r_ℓ . We first derive upper bounds on the probabilities of differential characteristics with active input/output bits of Hamming weight at most three, alongside upper bounds on the absolute correlations of linear trails with input masks of Hamming weight at most two. These theoretical bounds are detailed in Table 2. Based on these results, the viable parameter configurations for DL distinguishers satisfy $r_d \leq 6$ and $r_\ell \leq 8$, while those for boomerang distinguishers require $r_d^0 \leq 6$ and $r_d^1 \leq 6$. As for the number of round r_m of the middle part, we initialize the search at a conservative upper limit (e.g., $r_m = 10$) and iteratively decrement its value until a valid distinguisher is identified.

Table 2: Upper bounds of probabilities and absolute correlations

Rounds	Differential Probability		Linear Correlation
	Input HW ≤ 3	Output HW ≤ 3	Input HW ≤ 2
1	2^0	2^{-2}	2^0
2	2^{-2}	2^{-4}	2^{-1}
3	2^{-4}	2^{-8}	2^{-3}
4	2^{-8}	2^{-12}	2^{-6}
5	2^{-14}	2^{-18}	2^{-11}
6	2^{-26}	2^{-28}	2^{-12}
7	2^{-34}	2^{-40}	2^{-16}
8	—	—	2^{-20}

Experimental Results of the Differential-Linear Distinguisher. Our experimental results show that the longest DL distinguisher spans 17 rounds, with parameters $r_d = 4$, $r_m = 5$, and $r_\ell = 8$, and an absolute correlation of $2^{-59.2542}$. For the 4-round differential characteristic, the probability is 2^{-14} , while the absolute correlation of the 8-round linear trail is 2^{-20} . The absolute correlation of the middle part is $2^{-5.2542}$.

Specifically, we find a total of 342 4-round differential characteristics with probability 2^{-14} and 32 8-round linear trails with absolute correlation 2^{-20} . Considering all 342×32 combinations, we obtained 1156 5-round DL transitions for the middle part with absolute correlation $\geq 2^{-11}$. Among these DL transitions, we count the number of instances falling into different absolute correlation intervals. Specifically, 12 transitions lie in $[2^{-6}, 2^{-5})$, 45 in $[2^{-7}, 2^{-6})$,

279 in $[2^{-8}, 2^{-7})$, 524 in $[2^{-9}, 2^{-8})$, 292 in $[2^{-10}, 2^{-9})$, and 4 in $[2^{-11}, 2^{-10})$. Table 3 lists the input differences and output masks of the middle part of the DL distinguishers with high absolute correlations.

Table 3: Specification of partial DL distinguishers

Δ_m	Γ_m	$ p_{DL}^2 q_{DL}^2 $	r_{DL}
0x04000000200000000000000000000000	0x000000000000000000000000090000000000	2^{-54}	$2^{-5.2542}$
0x00000000000000000000000040000000200000	0x000090000000000000000000000000000000	2^{-54}	$2^{-5.2804}$
0x00040000002000000000000000000000	0x0000000000000000000000000900000000	2^{-54}	$2^{-5.3380}$
0x0000000000000000000000004000000020000	0x000009000000000000000000000000000000	2^{-54}	$2^{-5.3438}$
0x02000000100000000000000000000000	0x000000000000000000000000090000000000	2^{-54}	$2^{-5.4309}$
0x00000000000000000000000020000000100000	0x000090000000000000000000000000000000	2^{-54}	$2^{-5.4367}$
0x0000000000000000000000002000000010000	0x000009000000000000000000000000000000	2^{-54}	$2^{-5.4394}$
0x00020000001000000000000000000000	0x000000000000000000000000090000000000	2^{-54}	$2^{-5.4587}$
0x000004000000002000000000000000000000	0x000000000000000000000000090000000000	2^{-54}	$2^{-5.9224}$
0x000000000000000000000000000000000000	0x000009000000000000000000000000000000	2^{-54}	$2^{-5.9264}$
0x00000000000000000000000040000000200	0x000090000000000000000000000000000000	2^{-54}	$2^{-5.9376}$
0x000000004000000020000000000000000000	0x000000000000000000000000090000000000	2^{-54}	$2^{-5.9378}$
0x000000000000000000000000200000000100	0x000090000000000000000000000000000000	2^{-54}	$2^{-6.0295}$
0x000002000000010000000000000000000000	0x000000000000000000000000090000000000	2^{-54}	$2^{-6.0315}$
0x000000002000000010000000000000000000	0x000000000000000000000000090000000000	2^{-54}	$2^{-6.0572}$
0x0000000000000000000000002000000001	0x000009000000000000000000000000000000	2^{-54}	$2^{-6.0694}$

Table 4: Partial boomerang distinguishers with $[r_d, r_m, r_\ell] = [5, 6, 5]$

Δ_m^0	Δ_m^1	$ p_B^2 q_B^2 $	r_B
0x00000000000000000000000020000000100	0x000000000000004040000000000000000000	2^{-96}	$2^{-18.0242}$
0x00000000000000000000000000000000100	0x094000000000000000000000000000000000	2^{-96}	$2^{-18.0995}$
0x000000000000000000000000000000000004	0x00000000000000000000000094000000000000	2^{-96}	$2^{-18.1790}$
0x000000000000000000000000000000000000	0x0000000000000000000000000000000000004009	2^{-96}	$2^{-18.2065}$
0x000000000000010000000000000000000000	0x00000000000000000000000000000000000940	2^{-96}	$2^{-18.3523}$
0x000000000000010000000000000000000000	0x00000000000000000000000000000000000940	2^{-96}	$2^{-18.3523}$
0x000010000000000000000000000000000000	0x000000000000000000000000000000000004009	2^{-96}	$2^{-18.4150}$
0x0000000000000000000000000000000001000	0x00000000000000000000000000000009400000	2^{-96}	$2^{-18.4806}$
0x000000000000000000000000000000000000	0x0000000000000000000000000940000000000000	2^{-96}	$2^{-18.5670}$
0x0000000000000000000000000020000000100	0x00000000000000000000000004040000000000	2^{-96}	$2^{-18.5737}$
0x000000000000000000000000000000000000	0x000000000000040090000000000000000000	2^{-96}	$2^{-18.5805}$
0x000010000000000000000000000000000000	0x000040090000000000000000000000000000	2^{-96}	$2^{-18.5872}$
0x000001000000000000000000000000000000	0x000040090000000000000000000000000000	2^{-96}	$2^{-18.6215}$
0x000004000000000000000000000000000000	0x000040090000000000000000000000000000	2^{-96}	$2^{-18.6215}$
0x000000000000000000000000000000000000	0x000000000000000000000000040400000000	2^{-96}	$2^{-18.6590}$
0x000000100000000000000000000000000000	0x0000000000000000000000000400900000000	2^{-96}	$2^{-18.7776}$
0x00000000000000400000000000000000100	0x0000000000000000000000004040000000000	2^{-96}	$2^{-18.8625}$
0x000000000000000000000000000000000001	0x0000000000000000000000009400000000000	2^{-96}	$2^{-18.9527}$

Experimental Results of the Boomerang Distinguisher. Our experimental results show that the longest Boomerang distinguisher spans 16 rounds, with parameters $r_d^0 = 5$, $r_m = 6$, and $r_d^1 = 5$, and an overall probability of $2^{-114.024}$. For the 5-round upper differential characteristic, the probability is $p_B = 2^{-24}$, and for the 5-round lower differential characteristic, the probability is $q_B = 2^{-24}$. The probability of the middle part is $r_B = 2^{-18.024}$.

We identified 4230 combinations of 5-round upper and lower differential characteristics satisfying $2^{-52} \leq p_{BQ_B} \leq 2^{-48}$, yielding 1989 6-round middle-part boomerang transitions. The number of transitions exceeding specific probability thresholds is summarized as follows: 18 in $[2^{-115}, 2^{-113})$, 61 in $[2^{-119}, 2^{-115})$, 352 in $[2^{-120}, 2^{-119})$, 499 in $[2^{-121}, 2^{-120})$, 740 in $[2^{-122}, 2^{-121})$, 311 in $[2^{-123}, 2^{-122})$, and 8 in $[2^{-124}, 2^{-123})$. Table 4 lists the input differences and output differences of the middle part of the boomerang distinguishers with high probabilities.

The 16-round Boomerang distinguishers presented in Table 4 are unsuitable for key recovery, as extending them by one round in both forward and backward directions leads to an attack complexity exceeding that of exhaustive search. Consequently, we truncate these distinguishers by one round to obtain the 15-round Boomerang distinguishers in Table 5. These modified distinguishers permit a two-round forward and one-round backward extension for key recovery. Ultimately, we select the third distinguisher in Table 5, which achieves a probability of $2^{-90.179}$.

Table 5: Partial boomerang distinguishers with $[r_d, r_m, r_\ell] = [4, 6, 5]$ [illegible]

4.2 MILP Models for Searching Attacks

MILP Model for DL Attack. Following the 17-round DL distinguisher, we extend one round backward and one round forward, resulting in a 19-round DL key-recovery attack. Our model follows the framework proposed in [13]. For brevity, we present only the variable representations and the objective function in our model.

Variable Representation. Each differential or mask variable is encoded as a pair (a, b) : $(0, 0)$ represents an inactive difference or mask, $(0, 1)$ represents an active difference or mask, and $(1, 0)$ represents an unknown difference or mask.

Objective Function. Let (x_0^j, y_0^j) denote the output differential variables of S-box in the first round (backward extension), and (x_{18}^j, y_{18}^j) denote the input mask variables of S-box in the 19th round (forward extension). We define gk_0^j and gk_{18}^j as the number of key bits to be guessed in the first and last rounds, respectively. The objective function is defined as Minimize $\omega_0 \cdot \sum_{j=0}^{31} gk_0^j + \omega_{18} \cdot \sum_{j=0}^{31} gk_{18}^j$, where gk_0^i and gk_{18}^i are determined by

$$gk_0^i = \begin{cases} 0, & \text{if } [x_0^{4i+3}, y_0^{4i+3}, x_0^{4i+2}, y_0^{4i+2}, x_0^{4i+1}, y_0^{4i+1}, x_0^{4i}, y_0^{4i}] \\ & = [0, 0, 0, 0, 0, 0, 0, 0] \text{ or } = [0, 1, 0, 1, 0, 1, 0, 1], \\ 6, & \text{otherwise.} \end{cases}$$

$$gk_{18}^i = \begin{cases} 0, & \text{if } \sum_{k=0}^3 (x_{18}^{4i+k} + y_{18}^{4i+k}) = 0, \\ 2, & \text{if } \sum_{k=0}^3 (x_{18}^{4i+k} + y_{18}^{4i+k}) = 1 \text{ and } x_{18}^{4i+3} + y_{18}^{4i+3} = 1, \\ 8, & \text{if } \sum_{k=0}^3 (x_{18}^{4i+k} + y_{18}^{4i+k}) = 1 \text{ and } x_{18}^{4i+3} + y_{18}^{4i+3} \neq 1, \\ 16, & \text{otherwise.} \end{cases}$$

Here, ω_0 and ω_{18} are weighting parameters chosen by the attacker.

MILP Model for Boomerang Attack. The probability of our 16-round boomerang distinguisher is less than 2^{-104} . Extending the distinguisher by one round backward or forward introduces at least five active S-boxes, resulting in the guessing of at least 20 key bits. Even when applying the decision tree technique, approximately $6^5 \approx 2^{12.925}$ key guesses are required. According to Formula 4, the time complexity of the 17-round attack is at least $2^{129.925} > 2^{128}$. Therefore, we truncate the first round of the 16-round distinguisher to obtain a 15-round distinguisher, corresponding to a parameter $[r_d, r_m, r_\ell]$ adjustment from $[5, 6, 5]$ to $[4, 6, 5]$. By extending this distinguisher one round backward and two rounds forward, we obtain an 18-round boomerang attack. The variable representations in this model inherit directly from the DL counterpart. Thus, we solely present the objective function tailored for the 18-round attack to avoid redundancy.

Objective Function. According to Equation 4, the dominant term in the time complexity of our boomerang attack is given by $\frac{2^{k_b - n + 2m_f}}{P^2 \cdot \epsilon}$. Let η denote the computational cost of a partial decryption relative to a full 18-round encryption. The filtering cost coefficient can then be expressed as $\epsilon = 2^{k_f - m_f} \cdot \eta$. Consequently, the overall time complexity is dominated by the exponent $k_b + 2m_f + k_f - m_f = k_b + m_f + k_f$. Accordingly, the objective function of our automated MILP model is formulated as Minimize $k_b + m_f + k_f$.

Characterization of $k_b + k_f$. Let (x_0^j, y_0^j) denote the output differential variables of the j -th S-box in the first round, (x_{16}^j, y_{16}^j) denote the output differential variables of the j -th S-box in the 17th round, and (x_{17}^j, y_{17}^j) denote the input differential variables of the j -th S-box in the 18th round. We further define gk_0^j , gk_{17}^j , and gk_{18}^j as the numbers of guessed key bits involved in the whitening key, the 17th round, and the 18th round, respectively. Then, the

total number of guessed key bits $k_b + k_f$ can be characterized as $k_b + k_f = \sum_{j=0}^{127} \max \{gk_0^j, gk_{17}^{(j-17) \bmod 128}, gk_{18}^{(j-18) \bmod 128}\}$, where $gk_0^{4*i+j} = x_0^{4*i+j} + y_0^{4*i+j}$, $0 \leq i \leq 31, 0 \leq j \leq 2$, and $gk_i^j = x_i^j + y_i^j$, $17 \leq i \leq 18, 0 \leq j \leq 127$.

Characterization of m_f . We partition the input differential state of the 16th round, denoted by Δ_{16} , into eight independent 16-bit substates. Specifically, for each $0 \leq j \leq 7$, the j -th substate is represented as $(\Delta_{16}^{15+16j}, \Delta_{16}^{14+16j}, \dots, \Delta_{16}^{16j}) = M^j \in \{0, 1\}^{16}$. Suppose that the propagation of M^j through two rounds results in $2^{m_f^j}$ possible values of the 16-bit ciphertext difference Δ_C . Then, the parameter m_f can be characterized by summing the individual contributions $m_f = \sum_{j=0}^7 m_f^j$. Finally, the objective function of our MILP model is

$$\text{Minimize} \quad \sum_{j=0}^{127} \max \{gk_0^j, gk_{17}^{(j-17) \bmod 128}, gk_{18}^{(j-18) \bmod 128}\} + \sum_{j=0}^7 m_f^j.$$

5 Differential-based Attacks on BAKSHEESH

5.1 Differential-Linear Attack on 19-round BAKSHEESH

Our MILP model identifies the DL attack configuration $\Delta_P \xrightarrow{1 \text{ round}} \Delta_1 \xrightarrow{17 \text{ rounds}} \Gamma_{18} \xrightarrow{1 \text{ round}} \Gamma_C$, which is summarized in Table 6. Here, Δ_1 and Γ_{18} denote the input difference and output mask of the 17-round DL distinguisher, which exhibits an absolute correlation of $2^{-59.2542}$ (The middle part of the distinguisher is presented in the first row of Table 3). By extending Δ_1 backward by one round, we derive the difference after the first-round S-box layer, denoted by Δ_{S_0} , as well as the plaintext difference Δ_P . Similarly, extending Γ_{18} forward by one round yields the mask after the 19th-round S-box layer, denoted by $\Gamma_{S_{18}}$, and the corresponding ciphertext mask Γ_C . The symbols -, 1, and * indicate inactive, active, and unknown difference/mask bits, respectively. The forward and backward extension procedures involve the subkeys k_0 and k_{19} . The detailed attack procedure is presented in Algorithm 1. The key-guessing phase is divided into three parts, as shown in Table 7.

Table 6: The differential-linear attack on 19-round BAKSHEESH

Δ_P	1*1* ---- -1** ---- ---- ---- ---- ---- ---- ---- 1*** ---- ---- ---- ----
	---- ---- ---- ---- ---- ---- ---- ---- ---- ---- ---- ---- 1*** -1**
Δ_{S_0}	--1- ---- 1--- ---- ---- ---- ---- ---- ---- ---- ---- -1-- ---- ---- ---- ----
	---- ---- ---- ---- ---- ---- ---- ---- ---- ---- ---- ---- --1 1---
Δ_1	---- ---- ---- ---- ---- ---- ---- 1--1 1-1- ---- -1-- ---- ---- ---- ----
	---- ---- ---- ---- ---- ---- ---- ---- ---- ---- ---- ---- --1- --1- --1- --1-
Γ_{18}	---- -1 ---- ---- -1-- -1-- -1-- -1-- -1-- ---- ---- ---- --1- --1- --1- --1-
	--1 ---- ---- --1 --1 --1 1---- ---- ---- 1---- ---- ---- ---- ----
$\Gamma_{S_{18}}$	---- ---- ---- 1*** 1*** 1*** 1*** 1*** ---- ---- ---- **1* **1* **1* **1*
	**** ---- ---- ---- **** **** **** **** -111 ---- ---- -111 ---- ---- ----
Γ_C	1--- 1*** ---- **1* ---* ---- -1-- ---- -*-- 1*** ---- **1* --*- **** -11- ----
	--*- 1*** ---- **1* ---* ---- -11 ---- --*- 1*** ---- **1* *--- **** --1 ----

Algorithm 1 Key-Recovery Procedure of the 19-Round DL Attack

```

1: Data Collection
2: Choose  $2^{119.94}$  plaintext-ciphertext pairs
3: Guess-and-Filter
4: for each guess of subkey bits  $k_0^0, k_0^4, k_0^{80}, k_0^{116}, k_0^{124}$  do
5:   for each guess of subkey bits  $k_0^1, k_0^2, k_0^5, k_0^6, k_0^{81}, k_0^{82}, k_0^{117}, k_0^{118}, k_0^{125}, k_0^{126}$  do
6:     Evaluate the decision trees of S-boxes 0, 1, 20, 29, 31 for all plaintexts  $P_i$ 
7:     Derive plaintext pairs  $(P_i, P'_i)$  satisfying the input difference  $\Delta_P$ 
8:     Encrypt  $(P_i, P'_i)$  to obtain ciphertext pairs  $(C_i, C'_i)$ 
9:     for each guess of 44 bits of subkey bit  $k_{19}$  do
10:      Partially decrypt ciphertext pairs  $(C_i, C'_i)$  one round
11:      Compute correlation  $cor = \langle \Gamma_{18}, C_i \oplus C'_i \rangle$ 
12:      if  $cor$  matches the expected correlation then
13:        Store the current guess as a candidate key
14:      end if
15:    end for
16:  end for
17: end for
18: Exhaustive Search
19: Exhaustively search the remaining key bits together with candidate keys
20: return The correct master key

```

Table 7: Summary of guessed subkey bits for the key-recovery attack

Step	Guessed Subkey Bits / Linear Combinations
Lines 4 and 5	$k_0^0, k_0^4, k_0^{80}, k_0^{116}, k_0^{124}$ and $k_0^1, k_0^2, k_0^5 \oplus k_0^6, k_0^8, k_0^9 \oplus k_0^{10}, k_0^{11}, k_0^{12} \oplus k_0^{13}, k_0^{14}, k_0^{15} \oplus k_0^{16}, k_0^{17}, k_0^{18} \oplus k_0^{19}, k_0^{20}, k_0^{21} \oplus k_0^{22}, k_0^{23}, k_0^{24} \oplus k_0^{25}, k_0^{26}, k_0^{27} \oplus k_0^{28}, k_0^{29}, k_0^{30} \oplus k_0^{31}$ (see the decision trees in Appendix B (c)-(f))
Line 9 (44 bits)	$k_{19}^{61} \oplus k_{19}^{28}, k_{19}^{127} \oplus k_{19}^{94}, k_{19}^{127} \oplus k_{19}^{61}, k_{19}^{89} \oplus k_{19}^{56}, k_{19}^{27} \oplus k_{19}^{122}, k_{19}^{27} \oplus k_{19}^{89}, k_{19}^{121} \oplus k_{19}^{88}, k_{19}^{59} \oplus k_{19}^{26}, k_{19}^{59} \oplus k_{19}^{121}, k_{19}^{25} \oplus k_{19}^{120}, k_{19}^{91} \oplus k_{19}^{58}, k_{19}^{91} \oplus k_{19}^{25}, k_{19}^{57} \oplus k_{19}^{24}, k_{19}^{123} \oplus k_{19}^{90}, k_{19}^{123} \oplus k_{19}^{57}, k_{19}^{114} \oplus k_{19}^{48}, k_{19}^{19} \oplus k_{19}^{81}, k_{19}^{114} \oplus k_{19}^{81}, k_{19}^{18} \oplus k_{19}^{80}, k_{19}^{51} \oplus k_{19}^{113}, k_{19}^{18} \oplus k_{19}^{113}, k_{19}^{50} \oplus k_{19}^{112}, k_{19}^{83} \oplus k_{19}^{17}, k_{19}^{82} \oplus k_{19}^{16}, k_{19}^{115} \oplus k_{19}^{49}, k_{19}^{82} \oplus k_{19}^{49}, k_{19}^{110} \oplus k_{19}^{77}, k_{19}^{15} \oplus k_{19}^{110}, k_{19}^{110} \oplus k_{19}^{44}, k_{19}^{78} \oplus k_{19}^{45}, k_{19}^{111} \oplus k_{19}^{78}, k_{19}^{12} \oplus k_{19}^{106}, k_{19}^{73} \oplus k_{19}^{11}, k_{19}^{106} \oplus k_{19}^{40}, k_{19}^{10} \oplus k_{19}^{105}, k_{19}^{43} \oplus k_{19}^{10}, k_{19}^{10} \oplus k_{19}^{72}, k_{19}^{42} \oplus k_{19}^9, k_{19}^{75} \oplus k_{19}^{42}, k_{19}^{42} \oplus k_{19}^{104}, k_{19}^{74} \oplus k_{19}^{41}, k_{19}^8 \oplus k_{19}^{38}, k_{19}^5 \oplus k_{19}^{100}$.

Therefore, the backward extension by one round involves five active S-boxes. We adopt the decision tree key-guessing strategy, resulting in an overall key-guessing complexity of $6^5 \approx 2^{12.925}$. For the forward extension by one round, 44 key bits need to be guessed. Following [25] and [26], key recovery for the linear part is efficiently performed using the FFT technique [34]. Specifically, for each guess of the linear combinations of the last-round subkey bits, the computational cost consists of $2 \cdot 44 \cdot 2^{44}$ additions/subtractions and 2^{44} multiplications.

In this paper, the lower bound for a 19-round encryption is estimated as $128 \cdot 3 \cdot 19 = 7296$ bit operations. Assuming that a 128-bit addition requires approximately 256 bit operations and a 128-bit multiplication costs about $128 \cdot$

$\log_2 128 = 896$ bit operations, we obtain worst-case conversion factors of $2^{-4.83}$ for additions and $2^{-3.03}$ for multiplications. Accordingly, the time complexity of the key-recovery phase for the linear part is equivalent to $2 \cdot (2^{-3.03} + 2^{-4.83} \cdot 44) \cdot 2^{2 \cdot 44}$ 19-round encryptions.

Complexity Evaluation. According to Equation 1, when the advantage is set to 2 and the success probability is 0.55, the required number of plaintext pairs is $N = 2^{119.94}$, which implies that the data complexity of this attack is $D = 2N = 2^{120.94}$ plaintexts. The attack requires $D = 2^{120.94}$ registers to store the data and an additional 2^{44} memory units for the FFT distillation tables. Consequently, the total memory complexity is approximately $2^{120.94}$ blocks. In summary, the parameters of our 19-round DL attack are $N = 2^{119.94}$, $k_b = 12.925$, $\varepsilon = \frac{1}{19} \cdot \frac{5}{32}$, $\rho_M = 2^{-3.03}$, $\rho_A = 2^{-4.83}$, $k_f = 44$, $k = 128$, and $a = 2$. Thus, based on Equation 2, the overall time complexity of our 19-round DL attack on BAKSHEESH is estimated as

$$\begin{aligned} T &\approx N + 2^{k_b} \left(N\varepsilon + 2(\rho_M + \rho_A k_f) \cdot 2^{2k_f} \right) + 2^{k-a} \\ &= 2^{119.94} + 2^{12.925} \left(2^{119.94} \cdot \frac{1}{19} \cdot \frac{5}{32} + 2((2^{-3.03} + 2^{-4.83} \cdot 44) \cdot 2^{2 \cdot 44}) \right) + 2^{128-2} \\ &\approx 2^{126.97} \text{ 19-round encryptions.} \end{aligned}$$

5.2 Boomerang Attack on 18-round BAKSHEESH

Our MILP model identifies the boomerang attack configuration $\Delta_P \xrightarrow{1 \text{ round}} \Delta_1 \xrightarrow{15 \text{ rounds}} \Delta_{16} \xrightarrow{2 \text{ rounds}} \Delta_C$, which is summarized in Table 10. Here, Δ_1 and Δ_{16} denote the input and output differences of the 15-round boomerang distinguisher, which exhibits a probability of $2^{-90.179}$ (The middle part of the distinguisher is presented in the third row of Table 5). By extending Δ_1 backward by one round, we derive the difference after the first-round S-box layer Δ_{S_0} , as well as the plaintext difference Δ_P . Similarly, extending Δ_{16} forward by two rounds yields the differences after the 16th-round and 17th-round S-box layers, denoted by $\Delta_{S_{16}}$ and $\Delta_{S_{17}}$, and ciphertext difference Δ_C . The symbols -, 1, and * indicate inactive, active, and unknown difference bits, respectively. The forward and backward extension procedures involve the subkeys k_0 , k_{17} , and k_{18} . The attack procedure is presented in Algorithm 2.

According to Equation 4, the complexity of Algorithm 2 depends on several parameters, including the number of guessed key bits of k_0 , denoted by k_b , the logarithm of the number of all possible ciphertext differences m_f , and the time complexity ϵ required to update the counters for the guessed k_f bits. In the following, we detail these parameters in our attack.

- k_b . For the 15-round boomerang distinguisher, extending one round backward (i.e., Round 1) introduces the 11th, 12th, 25th, 27th, 29th, and 30th S-boxes, each involving three associated key bits. We directly guess the key bits $k_0^{44,45,46}$ corresponding to the 12th S-box. For the 11th, 25th, 27th, 29th, and 30th S-boxes, the corresponding subkey values are recovered using the decision tree

technique, requiring only six guesses per S-box (see Appendix B (a)-(d)). As shown in Table 8, the total number of key hypotheses involved in this backward extension phase is $2^{k_b} = 2^3 \times 6^5 = 62208 \approx 2^{15.925}$.

Algorithm 2 Key-Recovery Procedure of the 18-round Boomerang Attack

- 1: **Data Collection**
 - 2: Construct $2^{96.09}$ plaintext structures, each containing 2^{14} plaintexts
 - 3: In each structure, bits at positions $\{40, 41, 44, 45, 46, 97, 98, 104, 105, 113, 114, 116, 117, 118\}$ are exhaustively varied while all other bits remain fixed
 - 4: **Guess-and-Filter**
 - 5: **for** each guess of subkey bits of E_b (size k_b) **do**
 - 6: Initialize 2^{k_f} counters $V[k_f] \leftarrow 0$
 - 7: **for** each structure S_i **do**
 - 8: Evaluate the decision trees of S-boxes 11, 12, 25, 27, 29, 30
 - 9: Extract plaintext pairs (P_1, P_2) satisfying the difference Δ_P and form the set $\mathcal{S} = \{(P_1, C_1, P_2, C_2) \mid E_b(P_1) \oplus E_b(P_2) = \Delta_{in}\}$
 - 10: Insert all elements of $\mathcal{S}$ into a hash table $\mathcal{H}$ indexed by the $(n - m_f)$ inactive positions of Δ_C in both C_1 and C_2
 - 11: Randomly select collisions (C_1, C_2) and (C'_1, C'_2) sharing the same $2(n - m_f)$ -bit index and form quartets (C_1, C_2, C_3, C_4) , where $(C_3, C_4) = (C'_1, C'_2)$
 - 12: **end for**
 - 13: Determine key candidates related to E_f using the quartets and increase the corresponding counters
 - 14: Select the top $2^{k_f - a}$ hits in the counter as key candidates
 - 15: **end for**
 - 16: **Exhaustive Search**
 - 17: Exhaustively search the remaining key bits together with candidate keys
 - 18: **return** The correct master key
-

Table 8: Subkeys involved in Rounds 1, 17, and 18

Key	Guessed Subkey Bits k_i^j	Equivalent Master Key Bits k_i^j
$i = 0$	44, 45, 46 (Direct Guess)	44, 45, 46
$i = 0$	40, 41, 42, 96, 97, 98, 104, 105, 106, 112, 113, 114, 116, 117, 118 (Decision Tree)	40, 41, 42, 96, 97, 98, 104, 105, 106, 112, 113, 114, 116, 117, 118 (Note: The decision trees indicate that bits 40, 97, 104, 113, and 116 are always guessed directly.)
$i = 18$	99, 90, 49, 8, 107, 66, 57, 16, 115, 74, 33, 24, 123, 82, 41, 0, 35, 26, 113, 72, 43, 2, 121, 80, 51, 10, 97, 88, 59, 18, 105, 64, 3, 122, 81, 40, 11, 98, 89, 48, 19, 106, 65, 56, 27, 114, 73, 32, 103, 94, 53, 12, 111, 70, 61, 20, 119, 78, 37, 28, 127, 86, 45, 4, 71, 62, 21, 108, 79, 38, 29, 116, 87, 46, 5, 124, 95, 54, 13, 100, 39, 30, 117, 76, 47, 6, 125, 84, 55, 14, 101, 92, 63, 22, 109, 68	117, 108, 67, 26, 125, 84, 75, 34, 5, 92, 51, 42, 13, 100, 59, 18, 53, 44, 3, 90, 61, 20, 11, 98, 69, 28, 115, 106, 77, 36, 123, 82, 21, 12, 99, 58, 29, 116, 107, 66, 37, 124, 83, 74, 45, 4, 91, 50, 121, 112, 71, 30, 1, 88, 79, 38, 9, 96, 55, 46, 17, 104, 63, 22, 89, 80, 39, 126, 97, 56, 47, 6, 105, 64, 23, 14, 113, 72, 31, 118, 57, 48, 7, 94, 65, 24, 15, 102, 73, 32, 119, 110, 81, 40, 127, 86
$i = 17$	75, 42, 9, 104, 79, 46, 13, 108, 15, 110, 77, 44, 83, 50, 17, 112, 19, 114, 81, 48, 87, 54, 21, 116, 23, 118, 85, 52, 91, 58, 25, 120, 27, 122, 89, 56	92, 59, 26, 121, 96, 63, 30, 125, 32, 127, 94, 61, 100, 67, 34, 1, 36, 3, 98, 65, 104, 71, 38, 5, 40, 7, 102, 69, 108, 75, 42, 9, 44, 11, 106, 73

- k_f . The subkeys involved in Rounds 1, 17, and 18 are summarized in Table 8. The key-guessing procedure prioritizes the subkeys of Round 1, followed by

those of Rounds 18 and 17 in sequence. Consequently, redundant subkey bits that appear in multiple rounds are highlighted in red to indicate that they have already been guessed.

- m_f . Our 18-round boomerang attack is extended forward by two rounds. We partition the 128-bit difference Δ_{16} into eight independent 16-bit blocks. These initial difference segments are explicitly assigned as follows:

$$\begin{aligned} \Delta_{16}^{15,\dots,0} &= 0x0080, \Delta_{16}^{47,\dots,32} = 0x0010, \Delta_{16}^{79,\dots,64} = 0x2020, \Delta_{16}^{111,\dots,96} = 0x4040, \\ \Delta_{16}^{31,\dots,16} &= 0x0000, \Delta_{16}^{63,\dots,48} = 0x1010, \Delta_{16}^{95,\dots,80} = 0x2020, \Delta_{16}^{127,\dots,112} = 0x0000. \end{aligned}$$

As illustrated in Figure 2, we take $\Delta_{16}^{15,\dots,0} = 0x0080$ as an example to analyze the number of possible output differences that can arise after two rounds of propagation. The total number of possible values for the ciphertext difference tuple $(\Delta_C^{123,90,57,24}, \Delta_C^{115,82,49,16}, \Delta_C^{107,74,41,8}, \Delta_C^{99,66,33,0})$ is determined by the size of the product of the active S-box output difference sets, which is given by $2^{m_f^0} = |\mathcal{A}_3| \times |\mathcal{A}_2| \times |\mathcal{A}_1| \times |\mathcal{A}_0| = 4 \times 1 \times 4 \times 4 = 64$. Similarly, the total number of possible differential patterns for the remaining 16-bit blocks of Δ_{16} after two rounds of propagation can be derived in a similar manner, and the comprehensive results are summarized in Table 9.

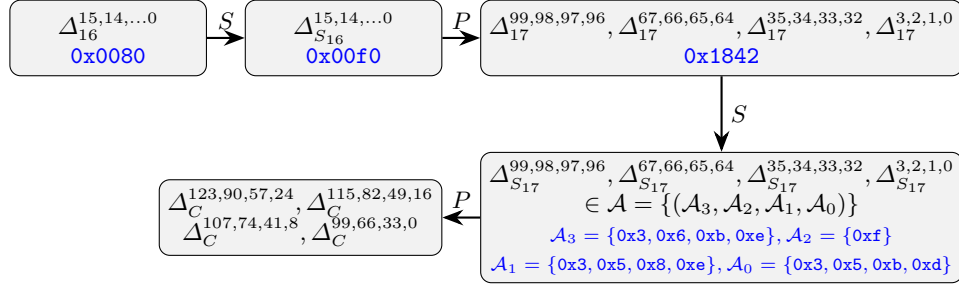


Fig. 2: Differential propagation over two rounds from Δ_{16} to Δ_C

Therefore, the logarithm of the number of all possible ciphertext differences m_f of our 18-round boomerang attack is $m_f = \sum_{j=0}^7 m_f^j = \log_2 64 + \log_2 1 + \log_2 64 + \log_2 2304 + \log_2 1600 + \log_2 1600 + \log_2 1009 + \log_2 1 \approx 54.44$.

- ϵ . The specific value of ϵ is determined by the following attack procedure:

- **Step 1:** Guess $k^{117,108,67,26}, k^{125,84,75,34}, k^{5,92,51,42}, k^{13,100,59,18}$ (16 bits in total). Decrypt to obtain $\Delta_{S_{16}}^{7,6,5,4}$. The cost is $2^{16} \cdot \frac{1}{18} \cdot \frac{4}{32} = 2^{8.83}$ with a filtering ratio of $1/64 = 2^{-6}$.
- **Step 2:** Guess $k^{53,44,3,90}, k^{61,20,11,98}, k^{69,28,115,106}, k^{77,36,123,82}$ (15 bits). Decrypt to obtain $\Delta_{S_{16}}^{36}$. The cost is $2^{10} \cdot 2^{15} \cdot \frac{1}{18} \cdot \frac{5}{32}$, where the filtering ratio is $1/64 = 2^{-6}$.
- **Step 3:** Guess $k^{21,12,99,58}, k^{29,116,107,66}, k^{37,124,83,74}, k^{45,4,91,50}$ and $k_{17}^{79} = k^{96}$ (15 bits). Decrypt to obtain $\Delta_{S_{16}}^{60,52}$. The cost is $2^{19} \cdot 2^{15} \cdot \frac{1}{18} \cdot \frac{6}{32}$ with a filtering ratio of $1/2304 = 2^{-11.17}$.

Table 9: Summary of 2-round differential propagation and entropy values m_f^j

j	Δ_{16}	Δ_C	m_f^j
0	$\Delta_{16}^{15,14,\dots,0} = 0\text{x}0080$	$\Delta_C^{99,90,49,8,107,66,57,16,115,74,33,24,123,82,41,0}$	$\log_2 64$
1	$\Delta_{16}^{31,30,\dots,16} = 0\text{x}0000$	$\Delta_C^{67,58,17,104,75,34,25,112,83,42,1,120,91,50,9,96}$	$\log_2 1$
2	$\Delta_{16}^{47,46,\dots,32} = 0\text{x}0010$	$\Delta_C^{35,26,113,72,43,2,121,80,51,10,97,88,59,18,105,64}$	$\log_2 64$
3	$\Delta_{16}^{63,62,\dots,48} = 0\text{x}1010$	$\Delta_C^{3,122,81,40,11,98,89,48,19,106,65,56,27,114,73,32}$	$\log_2 2304$
4	$\Delta_{16}^{79,78,\dots,64} = 0\text{x}2020$	$\Delta_C^{103,94,53,12,111,70,61,20,119,78,37,28,127,86,45,4}$	$\log_2 1600$
5	$\Delta_{16}^{95,94,\dots,80} = 0\text{x}2020$	$\Delta_C^{71,62,21,108,79,38,29,116,87,46,5,124,95,54,13,100}$	$\log_2 1600$
6	$\Delta_{16}^{111,110,\dots,96} = 0\text{x}4040$	$\Delta_C^{39,30,117,76,47,6,125,84,55,14,101,92,63,22,109,68}$	$\log_2 1009$
7	$\Delta_{16}^{127,126,\dots,112} = 0\text{x}0000$	$\Delta_C^{7,126,85,44,15,102,93,52,23,110,69,60,31,118,77,36}$	$\log_2 1$

- **Step 4:** Guess $k^{121,112,71,30}, k^{1,88,79,38}, k^{9,96,55,46}, k^{17,104,63,22}$ and $k_{17}^{81} = k^{98}$ (13 bits). Decrypt to obtain $\Delta S_{16}^{77,69}$. The cost is $2^{22.83} \cdot 2^{13} \cdot \frac{1}{18} \cdot \frac{6}{32}$ with a filtering ratio of $1/1600 = 2^{-10.644}$.
- **Step 5:** Guess $k^{89,80,39,126}, k^{97,56,47,6}, k^{105,64,23,14}, k^{113,72,31,118}$ (14 bits). Decrypt to obtain $\Delta S_{16}^{93,85}$. The cost is $2^{25.186} \cdot 2^{14} \cdot \frac{1}{18} \cdot \frac{6}{32}$ with a filtering ratio of $1/1600 = 2^{-10.644}$.
- **Step 6:** Guess $k^{57,48,7,94}, k^{65,24,15,102}, k^{73,32,119,110}, k^{81,40,127,86}, k_{17}^{25} = k^{42}$ and $k_{17}^{89} = k^{106}$ (15 bits). Decrypt to obtain $\Delta S_{16}^{110,102}$. The cost is $2^{28.542} \cdot 2^{15} \cdot \frac{1}{18} \cdot \frac{6}{32}$ with a filtering ratio of $1/1009 = 2^{-9.98}$.

The overall complexity factor ϵ is summarized as

$$\begin{aligned}
\epsilon = & \underbrace{2^{16} \cdot \frac{1}{18} \cdot \frac{4}{32}}_{\text{Step 1}} + \underbrace{2^{10} \cdot 2^{15} \cdot \frac{1}{18} \cdot \frac{5}{32}}_{\text{Step 2}} + \underbrace{2^{19} \cdot 2^{15} \cdot \frac{1}{18} \cdot \frac{6}{32}}_{\text{Step 3}} + \underbrace{2^{22.83} \cdot 2^{13} \cdot \frac{1}{18} \cdot \frac{6}{32}}_{\text{Step 4}} \\
& + \underbrace{2^{25.186} \cdot 2^{14} \cdot \frac{1}{18} \cdot \frac{6}{32}}_{\text{Step 5}} + \underbrace{2^{28.542} \cdot 2^{15} \cdot \frac{1}{18} \cdot \frac{6}{32}}_{\text{Step 6}} \approx 2^{37.034}.
\end{aligned}$$

Complexity Evaluation. According to Equation 3, when the expected number of surviving quartets s is set to 1 and the advantage a is set to 5, the success probability is 0.8413. The corresponding data complexity is $D = \sqrt{1} \cdot \frac{2^{128/2+1}}{2^{-90.179/2}} = 2^{110.09}$. The memory complexity is determined by the storage required for the hash tables during the quartet generation. Storing the encrypted pairs in a hash table requires approximately $M = D + D/2 + 2^{k_f} \approx 2^{110.09}$.

In summary, the parameters of our 18-round Boomerang attack are $s = 1$, $k_b = 15.925$, $n = 128$, $\text{Pr}^2 = 2^{-90.179}$, $m_f = 54.44$, $\epsilon = 37.034$, $k = 128$, and $a = 5$. Thus, based on Equation 4, the overall time complexity of our 18-round Boomerang attack on BAKSHEESH is estimated as

$$\begin{aligned}
T & \approx \sqrt{s} 2^{k_b+n/2+1} / \text{Pr} + s 2^{k_b-n+2m_f} / \text{Pr}^2 \cdot \epsilon + 2^{k-a} \\
& = 2^{15.925+128/2+1} \cdot 2^{90.179/2} + 2^{15.925-128+2 \cdot 54.44} \cdot 2^{90.179} \cdot 2^{37.034} + 2^{128-5} \\
& \approx 2^{126.48}.
\end{aligned}$$

[illegible]

In this paper, we integrate decision tree techniques with the two-round differential propagation property of BAKSHEESH to parameterize the guessed key bits and the logarithm of the number of possible plaintext/ciphertext differences. This motivates the design of an automated framework that accounts for the differential properties of non-linear components and searches for the best DL and boomerang key-recovery attacks under the stated constraints. Using this framework, we derive the first 19-round DL attack and the first 18-round boomerang attack with time complexities of $2^{126.97}$ and $2^{126.48}$, respectively. Additionally, based on an 18-round differential distinguisher, we construct the first 20-round chosen-ciphertext differential attack on BAKSHEESH with a time complexity of $2^{122.59}$, improving upon the previous best differential attack by one round.

References

1. Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. *J. Cryptol.* **4**(1), 3-72 (1991)
2. Matsui, M.: Linear cryptanalysis method for DES cipher. In: Hellesest, T. (ed.) EUROCRYPT'93. LNCS, vol. 765, pp. 386-397. Springer, Heidelberg (1994)
3. Wagner, D.A.: The boomerang attack. In: Knudsen, L.R. (ed.) FSE'99. LNCS, vol. 1636, pp. 156-170. Springer, Heidelberg (1999)
4. Langford, S.K., Hellman, M.E.: Differential-linear cryptanalysis. In: Desmedt, Y. (ed.) CRYPTO'94. LNCS, vol. 839, pp. 17-25. Springer, Heidelberg (1994)
5. Dunkelman, O., Keller, N., Shamir, A.: A practical-time related-key attack on the KASUMI cryptosystem used in GSM and 3G telephony. *J. Cryptol.* **27**(4), 824-849 (2014)

6. Bar-On, A., Dunkelman, O., Keller, N., et al.: DLCT: A new tool for differential-linear cryptanalysis. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part I. LNCS, vol. 11476, pp. 313-342. Springer, Cham (2019)
7. Zhao, Q., Fan, T., Wei, Y.: MILP-based differential cryptanalysis of the FBC-128 lightweight cipher. *Acta Electronica Sinica* **52**(6), 1896-1902 (2024)
8. Liu, S., Ren, X., Wang, S., et al.: Linear analysis of lightweight cipher ACE and SPIX based on mixed-integer linear programming. *Acta Electronica Sinica* **52**(9), 3065-3074 (2024)
9. Han, Y., Wang, C., Niu, Z., et al.: SAT-based automatic searching for differential and linear trails: Applying to CRAX. *Chinese Journal of Electronics* **33**(1), 72-79 (2024)
10. Delaune, S., Derbez, P., Vavrille, M.: Catching the fastest boomerangs: Application to SKINNY. *IACR Trans. Symmetric Cryptol.* **2020**(4), 104-129 (2020)
11. Bellini, E., G rault, D., Grados, J., et al.: Fully automated differential-linear attacks against ARX ciphers. In: Rosulek, M. (ed.) CT-RSA 2023. LNCS, vol. 13858, pp. 252-276. Springer, Cham (2023)
12. Hadipour, H., Nageler, M., Eichlseder, M.: Throwing boomerangs into feistel structures: Application to CLEFIA, WARP, LBlock, LBlock-s and TWINE. *IACR Trans. Symmetric Cryptol.* **2022**(3), 271-302 (2022)
13. Hadipour, H., Derbez, P., Eichlseder, M.: Revisiting differential-linear attacks via a boomerang perspective with application to AES, Ascon, CLEFIA, SKINNY, PRESENT, KNOT, TWINE, WARP, LBlock, Simeck, and SERPENT. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part IV. LNCS, vol. 14923, pp. 38-72. Springer, Cham (2024)
14. Liu, M., Hou, C., Lu, X., et al.: Differential-linear cryptanalysis from an algebraic perspective. *J. Cryptol.* **39**(1), 7 (2026)
15. Peng, T., Zhang, W., Weng, J., et al.: New approaches for estimating the bias of differential-linear distinguishers. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part IV. LNCS, vol. 14923, pp. 174-205. Springer, Cham (2024)
16. Hu, K., Niu, Z., Wang, M.: Round-based approximation of (higher-order) differential-linear correlation - A geometric approach perspective. In: Daemen, J., Thom , E. (eds.) EUROCRYPT 2026, Part VI. LNCS, vol. 15612, pp. 152-180. Springer, Cham (2026)
17. Chen, Y., Bao, Z., Yu, H.: Differential-linear approximation semi-unconstrained searching and partition tree: Application to LEA and Speck. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part III. LNCS, vol. 14440, pp. 223-255. Springer, Singapore (2023)
18. Gong, X., Wang, Q., Hao, Y., et al.: Persistence of hourglass(-like) structure: Improved differential-linear distinguishers for several ARX ciphers. In: Hanaoka, G., Yang, B.-Y. (eds.) ASIACRYPT 2025, Part I. LNCS, vol. 15467, pp. 317-351. Springer, Singapore (2025)
19. Hadipour, H., Sadeghi, S., Eichlseder, M.: Finding the impossible: Automated search for full impossible-differential, zero-correlation, and integral attacks. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part IV. LNCS, vol. 14007, pp. 128-157. Springer, Cham (2023)
20. Boura, C., Naya-Plasencia, M., Suder, V.: Scrutinizing and improving impossible differential attacks: Applications to CLEFIA, Camellia, LBlock and Simon. In: Sarkar, P., Iwata, T. (eds.) ASIACRYPT 2014, Part I. LNCS, vol. 8873, pp. 179-199. Springer, Heidelberg (2014)

21. Dong, X., Qin, L., Sun, S., et al.: Key guessing strategies for linear key-schedule algorithms in rectangle attacks. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part III. LNCS, vol. 13276, pp. 3-33. Springer, Cham (2022)
22. Song, L., Zhang, N., Yang, Q., et al.: Optimizing rectangle attacks: A unified and generic framework for key recovery. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part I. LNCS, vol. 13791, pp. 410-440. Springer, Cham (2022)
23. Song, L., Yang, Q., Chen, Y., et al.: Probabilistic extensions: A one-step framework for finding rectangle attacks and beyond. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part I. LNCS, vol. 14651, pp. 339-367. Springer, Cham (2024)
24. Broll, M., Canale, F., Flórez-Gutiérrez, A., et al.: Generic framework for key-guessing improvements. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part I. LNCS, vol. 13090, pp. 453-483. Springer, Cham (2021)
25. Broll, M., Canale, F., David, N., et al.: New attacks from old distinguishers: Improved attacks on Serpent. In: Galbraith, S.D. (ed.) CT-RSA 2022. LNCS, vol. 13161, pp. 484-510. Springer, Cham (2022)
26. Sabaly, T.M., Minier, M.: Differential-linear attacks from new distinguishers. *Des. Codes Cryptogr.* **94**(2), 37 (2026)
27. Wang, N., Wang, X., Jia, K., et al.: Differential attacks on reduced SIMON versions with dynamic key-guessing techniques. *Sci. China Inf. Sci.* **61**(9), 098103 (2018)
28. Qiao, K., Hu, L., Sun, S.: Differential analysis on Simeck and SIMON with dynamic key-guessing techniques. In: Camp, O., Furnell, S., Mori, P. (eds.) ICISSP 2016, CCIS, vol. 597, pp. 64-85. Springer, Cham (2016)
29. Liu, J., Tan, L., Xu, H.: New related-tweakey boomerang attacks and distinguishers on Deoxys-BC. *Chinese Journal of Electronics* **33**(3), 683-693 (2024)
30. Banik, S., Pandey, S.K., Peyrin, T., et al.: GIFT: A small Present. In: Fischer, W., Homma, N. (eds.) CHES 2017. LNCS, vol. 10529, pp. 321-345. Springer, Cham (2017)
31. Xu S., Chen S., Feng X., et al.: Cryptanalysis of BAKSHEESH block cipher. In: LIN D., Wang M., Yung M. (eds.) *Inscrypt 2024, Part II*. LNCS, vol. 15544. Springer, Cham, 2024: 382-402 (2024)
32. Lv Y., Zhang Y., Chen Z., et al.: Optimized linear cryptanalysis of BAKSHEESH: A case study with partial-sum, FWT, and key-bridging. *Frontiers of Computer Science* (2026)
33. Blondeau, C., Nyberg, K.: Improved parameter estimates for correlation and capacity deviates in linear cryptanalysis. *IACR Trans. Symmetric Cryptol.* **2016**(2), 162-191 (2016)
34. Flórez-Gutiérrez, A., Naya-Plasencia, M.: Improving key-recovery in linear attacks: Application to 28-round PRESENT. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part I. LNCS, vol. 12105, pp. 221-249. Springer, Cham (2020)
35. Selçuk, A.A.: On probability of success in linear and differential cryptanalysis. *J. Cryptol.* **21**(1), 131-147 (2008)
36. Baksi, A., Breier, J., Chattopadhyay, A., et al.: BAKSHEESH: Similar yet different from GIFT. *IACR Cryptol. ePrint Arch.* 2023, 750 (2023)
37. Baksi, A., Breier, J., Chattopadhyay, A., et al.: BAKSHEESH: Similar yet different from GIFT (and ZORRO). *IACR Commun. Cryptol.* **2**(4), 31 (2026)
38. Zhao, B., Dong, X., Jia, K.: New related-tweakey boomerang and rectangle attacks on Deoxys-BC including BDT effect. *IACR Trans. Symmetric Cryptol.* **2019**(3), 121-151 (2019)
39. Zhao, B., Dong, X., Meier, W., et al.: Generalized related-key rectangle attacks on block ciphers with linear key schedule: Applications to SKINNY and GIFT. *Des. Codes Cryptogr.* **88**(6), 1103-1126 (2020)

40. Sun, L., Wang, W., Wang M.: Accelerating the search of differential and linear characteristics with the SAT method. IACR Trans. Symmetric Cryptol. **2021**(1), 269-315 (2021)

A Differential Attack on 20-round BAKSHEESH

By employing existing automated search techniques [40], we identify an 18-round differential distinguisher for BAKSHEESH with a probability of 2^{-120} , denoted as $\Delta_2 \xrightarrow{18\text{-round}} \Delta_C$, where

$$\begin{aligned}\Delta_2 &= 0x400c09000000000000000000c4000000, \\ \Delta_C &= 0x22002600110013008800890044004c00.\end{aligned}$$

Then, we extend the distinguisher backward by two rounds to obtain a 20-round differential attack, denote as $\Delta_P \xrightarrow{2\text{-round}} \Delta_2 \xrightarrow{18\text{-round}} \Delta_C$. The details of the resulting 20-round differential attack on BAKSHEESH are summarized in Table 11. Similar to the 18-round boomerang attack, we begin by detailing the core parameters m_b and ϵ . It is worth noting that this 20-round differential attack is a chosen-ciphertext attack.

Table 11: The differential attack on 20-round BAKSHEESH

Δ_P	****	****	****	-1**	****	****	****	-1**	----	----	----	----	----	----	----	----	----	----	----
	****	****	-1**	****	****	****	****	-1**	****	-1**	****	****	****	****	****	****	1***	****	****
Δ_{S_0}	-*-	-*-	----	1---	----	----	----	1---	----	----	----	----	----	----	----	----	----	----	----
	-*-	----	1---	----	----	----	----	1---	----	1---	----	-*-	-*-	----	----	----	-1---	-1---	----
Δ_1	1***	1***	----	----	----	1***	----	----	----	----	----	----	----	1***	----	----	----	----	-1**
	----	----	----	----	----	1***	-1**	----	----	----	----	----	----	----	----	----	----	----	----
Δ_{S_1}	11--	-1--	----	----	----	-1--	----	----	----	----	----	----	----	-1--	----	----	----	1---	----
	----	----	----	----	----	----	-1	1---	----	----	----	----	----	----	----	----	----	----	----
Δ_2	-1--	----	----	11--	----	1--1	----	----	----	11--	-1--	----	----	----	----	----	----	----	----
	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----
Δ_{20}	--1-	-1--	----	----	--1-	-11-	----	----	--1	--1	----	----	----	--1	--11	----	----	----	----
	1---	1---	----	----	1---	1--1	----	----	-1--	-1--	----	----	----	-1--	11--	----	----	----	----

The subkeys involved in Rounds 1 and 2 are $\{k^{4i+3, 4i+2, 4i+1, 4i} \mid 0 \leq i \leq 15\} \cup \{k^{4i+3, 4i+2, 4i+1, 4i} \mid 24 \leq i \leq 31\}$, together with $k^{36,35,34,33}$, $k^{40,39,38,37}$, $k^{68,67,66,65}$, $k^{80,79,78,77}$, $k^{108,107,106,105}$, $k^{124,123,122,121}$, $k^{0,127,126,125}$.

- m_b . The 128-bit difference Δ_{S_1} can be decomposed into eight independent 16-bit blocks, whose corresponding entropy values m_b^j are summarized in Table 12. The logarithm of the number of all possible plaintext differences m_b of our 20-round differential attack is

$$\log_2 1760 + \log_2 160 + \log_2 160 + \log_2 160 + \log_2 160 + \log_2 1 + \log_2 160 + \log_2 304 \approx 48.317.$$

- ϵ . The value of ϵ is determined by the following six-step attack procedure:

- **Step 1:** Guess $k_0^{127,126,125,124}$, $k_0^{123,122,121,120}$, $k_0^{119,118,117,116}$, $k_0^{115,114,113,112}$.
 $k_1^{127,126,125,124} = k_0^{0,127,126,125}$ (17 bits in total). Encrypt to obtain $\Delta_{S_1}^{127,126,125,124}$.
The cost is $2^{17} \times \frac{1}{20} \times \frac{5}{32} = 2^{10}$ with a filtering ratio of $1/1760 \approx 2^{-10.782}$.

- **Step 2:** Guess $k_0^{111,110,109,108}, k_0^{107,106,105,104}, k_0^{103,102,101,100}, k_0^{99,98,97,96}, k_1^{123,122,121,120} = k_0^{124,123,122,121}$ (16 bits in total). Encrypt to obtain $\Delta_{S_1}^{123,122,121,120}$. The cost is $2^{6.218} \times 2^{16} \times \frac{1}{20} \times \frac{5}{32} = 2^{15.218}$ with a filtering ratio of $1/160 \approx 2^{-7.322}$.
- **Step 3:** Guess $k_0^{63,62,61,60}, k_0^{59,58,57,56}, k_0^{55,54,53,52}, k_0^{51,50,49,48}, k_1^{79,78,77,76} = k_0^{80,79,78,77}$, (20 bits). Encrypt to obtain $\Delta_{S_1}^{79,78,77,76}$. The cost is $2^{14.896} \times 2^{20} \times \frac{1}{20} \times \frac{5}{32} = 2^{27.896}$ with a filtering ratio of $1/160 \approx 2^{-7.322}$.
- **Step 4:** Guess $k_0^{47,46,45,44}, k_0^{43,42,41,40}, k_0^{39,38,37,36}, k_0^{35,34,33,32}, k_1^{107,106,105,104} = k_0^{108,107,106,105}$, (16 bits). Encrypt to obtain $\Delta_{S_1}^{51,50,49,48}$ and $\Delta_{S_1}^{107,106,105,104}$. The cost is $2^{27.574} \times 2^{16} \times \frac{1}{20} \times \frac{5}{32} = 2^{36.574}$ with a filtering ratio of $1/160 \approx 2^{-7.322}$.
- **Step 5:** Guess $k_0^{31,30,29,28}, k_0^{27,26,25,24}, k_0^{23,22,21,20}, k_0^{19,18,17,16}, k_1^{39,38,37,36} = k_0^{40,39,38,37}$ (16 bits). Encrypt to obtain $\Delta_{S_1}^{39,38,37,36}$. The cost is $2^{36.252} \times 2^{16} \times \frac{1}{20} \times \frac{5}{32} = 2^{45.252}$ with a filtering ratio of $1/160 \approx 2^{-7.322}$.
- **Step 6:** Guess $k_0^{15,14,13,12}, k_0^{11,10,9,8}, k_0^{7,6,5,4}, k_0^{3,2,1,0}, k_1^{67,66,65,64} = k_0^{68,67,66,65}, k_1^{35,34,33,32} = k_0^{30,35,34,33}$ (19 bits). Encrypt to obtain $\Delta_{S_1}^{99,98,97,96}$. The cost is $2^{44.93} \times 2^{19} \times \frac{1}{20} \times \frac{6}{32} = 2^{57.19}$ with a filtering ratio of $1/304 \approx 2^{-8.248}$.

The overall complexity factor ϵ is summarized as

$$\begin{aligned} \epsilon = & \underbrace{2^{17} \cdot \frac{1}{20} \cdot \frac{5}{32}}_{\text{Step 1}} + \underbrace{2^{6.218} \cdot 2^{16} \cdot \frac{1}{20} \cdot \frac{5}{32}}_{\text{Step 2}} + \underbrace{2^{14.896} \cdot 2^{20} \cdot \frac{1}{20} \cdot \frac{5}{32}}_{\text{Step 3}} + \underbrace{2^{27.574} \cdot 2^{16} \cdot \frac{1}{20} \cdot \frac{5}{32}}_{\text{Step 4}} \\ & + \underbrace{2^{36.252} \cdot 2^{16} \cdot \frac{1}{20} \cdot \frac{5}{32}}_{\text{Step 5}} + \underbrace{2^{44.93} \cdot 2^{19} \cdot \frac{1}{20} \cdot \frac{6}{32}}_{\text{Step 6}} \approx 2^{57.193}. \end{aligned}$$

Table 12: Summary of backward differential propagation and corresponding m_b^j

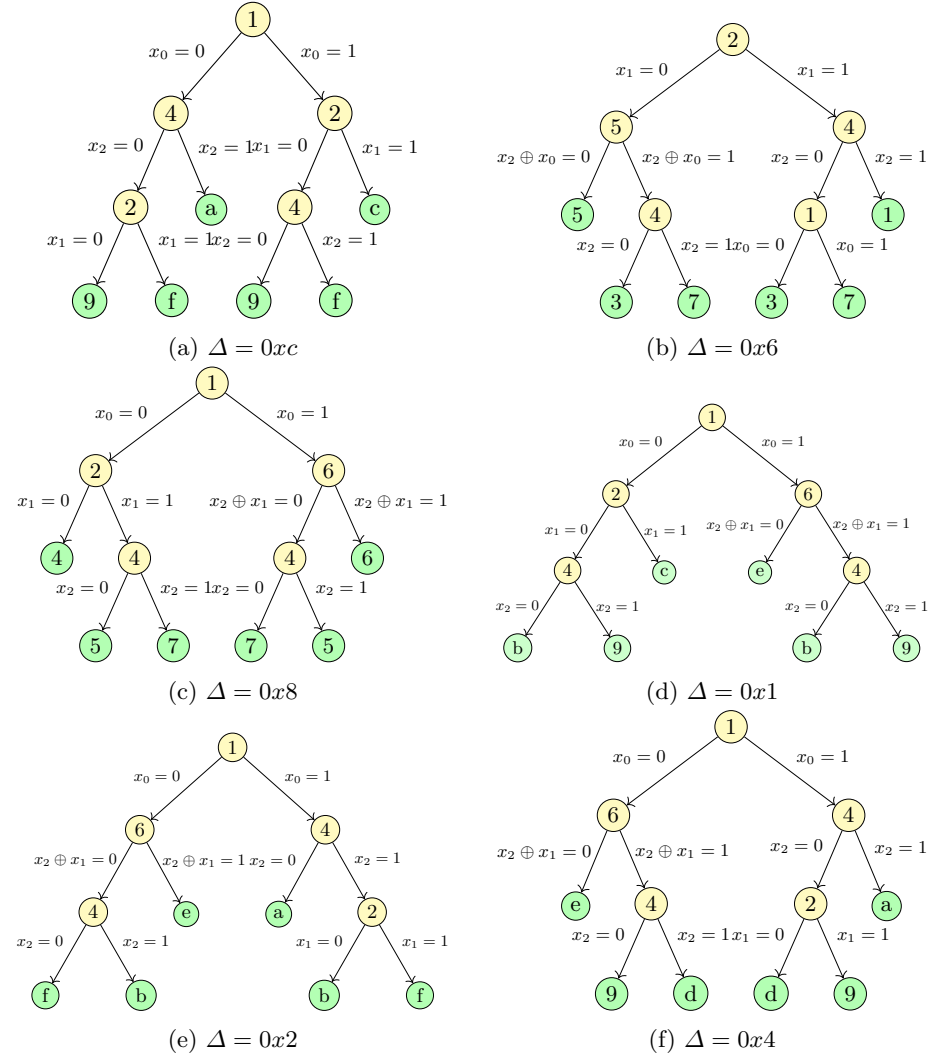
j	Δ_{S_1}	Δ_P	m_b^j
0	$\Delta_{S_1}^{99,98,97,96,67,66,65,64,35,34,33,32,3,2,1,0} = \text{0x0880}$	$\Delta_P^{15,14,\dots,0}$	$\log_2 1760$
1	$\Delta_{S_1}^{103,102,101,100,71,70,69,68,39,38,37,36,7,6,5,4} = \text{0x0010}$	$\Delta_P^{31,30,\dots,16}$	$\log_2 160$
2	$\Delta_{S_1}^{107,106,105,104,75,74,73,72,43,42,41,40,11,10,9,8} = \text{0x4000}$	$\Delta_P^{47,46,\dots,32}$	$\log_2 160$
3	$\Delta_{S_1}^{111,110,109,108,79,78,77,76,47,46,45,44,15,14,13,12} = \text{0x0400}$	$\Delta_P^{63,62,\dots,48}$	$\log_2 160$
4	$\Delta_{S_1}^{115,114,113,112,83,82,81,80,51,50,49,48,19,18,17,16} = \text{0x0000}$	$\Delta_P^{79,78,\dots,64}$	$\log_2 1$
5	$\Delta_{S_1}^{119,118,117,116,87,86,85,84,55,54,53,52,23,22,21,20} = \text{0x0000}$	$\Delta_P^{95,94,\dots,80}$	$\log_2 1$
6	$\Delta_{S_1}^{123,122,121,120,91,90,89,88,59,58,57,56,27,26,25,24} = \text{0x4000}$	$\Delta_P^{111,110,\dots,96}$	$\log_2 160$
7	$\Delta_{S_1}^{127,126,125,124,95,94,93,92,63,62,61,60,31,30,29,28} = \text{0xc000}$	$\Delta_P^{127,126,\dots,112}$	$\log_2 304$

Complexity Evaluation. The data complexity of our 20-round attack is $D = 2^{122}$ chosen ciphertexts. According to Equation 3, setting the expected number of surviving pairs to $s = 1$ and the advantage to $a = 7$ yields a success probability of 0.8015. Since $m_b = 48.317$ and $\epsilon = 57.193$, the time complexity is

$$T = D + \frac{D}{2^{n-m_b}} \cdot \epsilon + 2^{128-a} = 2^{122} + 2^{122-128+48.317+57.193} + 2^{128-7} \approx 2^{122.59}$$

of 20-round encryptions. The memory complexity is dominated by storing the round key counters, which requires 2^{104} memory blocks.

B Decision Trees for $\Delta \in \{0xc, 0x6, 0x8, 0x1, 0x2, 0x4\}$

Fig. 3: Decision trees for $\Delta \in \{0xc, 0x6, 0x8, 0x1, 0x2, 0x4\}$